

DOSSIER TECHNIQUE

BTS CIEL – Option IR

Épreuve E6-2 – Projet Technique

PROJET CERBÈRE

Système de Contrôle d'Accès Intelligent

SOMMAIRE

Partie commune	3
Présentation du projet	3
Expression du besoin	5
Modélisation	7
Diagramme de contexte	7
Diagramme de cas d'utilisation	8
Diagramme de déploiement	9
Schéma synoptique	10
Planning prévisionnel des tâches – Diagramme de GANTT	11
Nomenclature	12
Ressources du Projet	13
Ressources matérielles	13
Ressources logicielles	14
Prise en main du matérielles	15
Cahier de recette	17
Partie Personnelle	18
Modélisation	20
Diagramme de cas d'utilisation	20
Diagramme de déploiement	20
Diagramme de classe	21
Programme	23
Tests et prototypages	25
Conclusion	26

PARTIE COMMUNE

1. Présentation Projet CERBÈRE

1.1 Contexte de réalisation

Des travaux récents au LGT Joseph Gaillard ont conduit à un réaménagement complet des espaces de formation (salles et laboratoires). Ces espaces, dédiés à la conception et à la réalisation, sont destinés à accueillir un public varié : élèves, étudiants et enseignants de l'établissement, mais aussi des professionnels des entreprises partenaires du lycée, ainsi que des élèves et professeurs d'autres établissements scolaires.

Face à l'ouverture de ces espaces à des publics extérieurs, la direction de l'établissement a identifié un besoin urgent de sécurisation et d'automatisation de l'accès. Le Directeur délégué du LGT Joseph Gaillard a donc initié le projet CERBÈRE afin de répondre à cette problématique.

1.2 Présentation générale du projet

Le projet CERBÈRE est un système de contrôle d'accès intelligent développé pour sécuriser l'entrée des salles et laboratoires du LGT Joseph Gaillard. Son nom fait référence au gardien des portes de l'Enfer dans la mythologie grecque, illustrant ainsi sa vocation première : surveiller et contrôler l'accès.

CERBÈRE repose sur l'automatisation de la commande d'ouverture de deux types d'équipements de sécurité :

- Une porte équipée d'une gâche électrique (entrée de la salle W13)
- Une porte équipée d'une ventouse magnétique (entrée du laboratoire)

Le système s'articule autour de quatre grandes fonctions, chacune prise en charge par un étudiant :

- Étudiant 1 – Prendre en compte la présence d'un visiteur au laboratoire
- Étudiant 2 – Gérer la base de données d'identification et les accès
- Étudiant 3 – Visualiser in situ et à distance via une application mobile
- Étudiant 4 – Autoriser l'accès par badge RFID à la salle W13

2. Expression du Besoin

2.1 Problématique

Le LGT Joseph Gaillard ouvre ses espaces de formation (salles et laboratoires) à des publics diversifiés, incluant des visiteurs extérieurs à l'établissement. Cette ouverture soulève une problématique majeure : comment garantir la sécurité et la traçabilité des accès à ces espaces tout en offrant un système d'entrée fluide et automatisé ?

Le besoin exprimé par le commanditaire peut se résumer ainsi :

« Automatiser et sécuriser l'accès aux espaces de formation du LGT Joseph Gaillard en contrôlant la commande d'ouverture de portes équipées d'une gâche électrique ou d'une ventouse magnétique, tout en assurant la traçabilité des entrées et la protection des données. »

2.2 Fonctions principales du système

Fonction 1 – Prendre en compte la présence d'un visiteur (Étudiant 1)

Quand un visiteur se présente à l'entrée du laboratoire, le système doit :

- Détecter automatiquement une présence humaine via un capteur de proximité (PIR)
- Permettre au visiteur d'actionner une sonnette pour alerter les occupants
- Diffuser une sonnerie à l'intérieur du laboratoire
- Permettre l'ouverture de la porte à ventouse magnétique sur autorisation
- Déclencher une alerte en cas de tentative de sortie non autorisée

Fonction 2 – Gérer la demande d'accès et la base de données (Étudiant 2)

Le système doit disposer d'un backend sécurisé capable de :

- Modéliser et héberger une base de données MySQL centralisant les informations d'identification (photos, identifiants, mots de passe, rôles)
- Restreindre l'accès aux données aux seules personnes autorisées
- Garantir la confidentialité et l'intégrité des données via des mécanismes de chiffrement
- Fournir un site web administrateur permettant de superviser les accès et d'enregistrer de nouveaux utilisateurs
- Proposer un tableau de bord visuel affichant l'état des entrées en temps réel
- Permettre l'ajout et la suppression d'utilisateurs dans le système

Fonction 3 – Visualiser in situ et à distance (Étudiant 3)

Pour permettre une supervision à distance, le système doit :

- Proposer une application mobile intuitive
- Afficher en temps réel la présence ou l'absence d'utilisateurs dans les espaces
- Envoyer des alertes push en cas de présence suspecte ou non autorisée
- Chiffrer toutes les communications entre les composants (transceivers, base de données, application)
- Permettre la visualisation des flux vidéo (images horodatées)
- Notifier toute anomalie de manière proactive

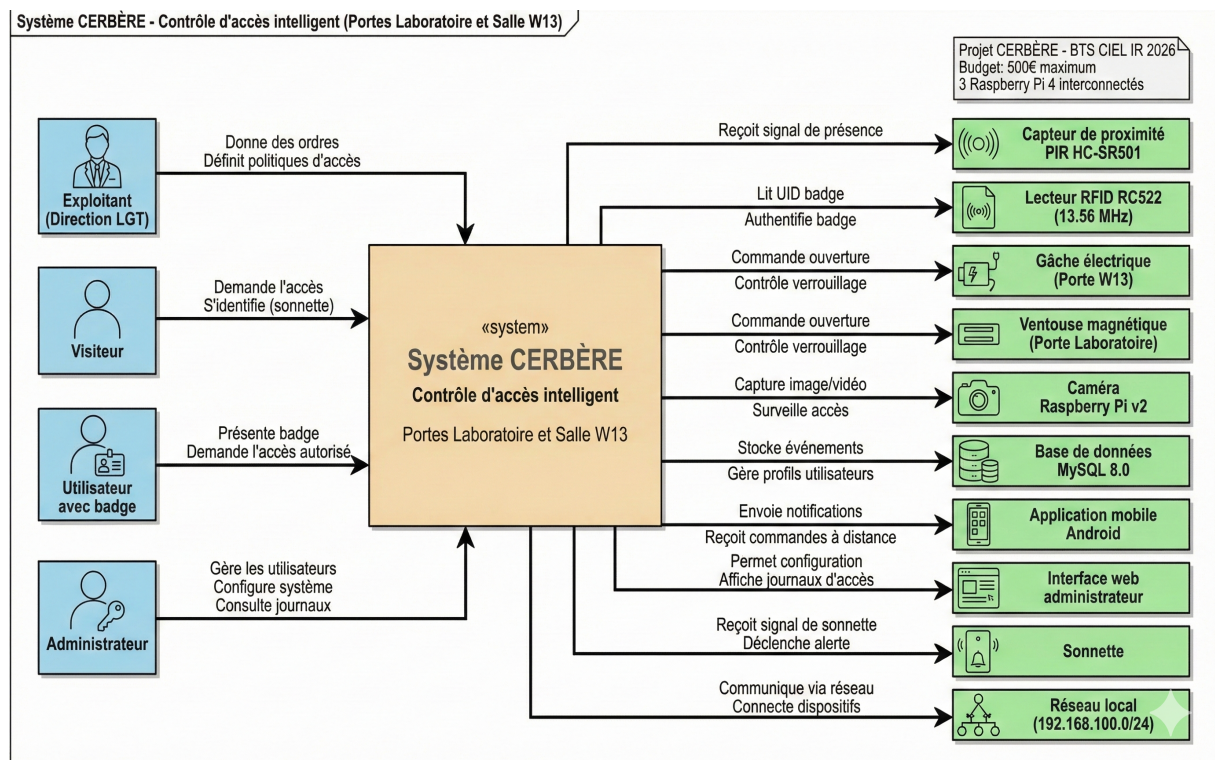
Fonction 4 – Autoriser l'accès par badge RFID (Étudiant 4)

Pour l'accès à la salle W13, le système doit :

- Identifier les visiteurs autorisés grâce à un badge RFID codé
- Contrôler la validité de l'autorisation en interrogeant la base de données
- Commander l'ouverture de la gâche électrique si l'autorisation est valide
- Installer un portail RFID (transceiver) à l'entrée de la salle W13
- Stocker les données de passage (déplacements, horaires) dans la base de données sécurisée
- Alerter en cas de tentative d'accès non autorisé

3.Modélisation

3.1 Diagramme de contexte

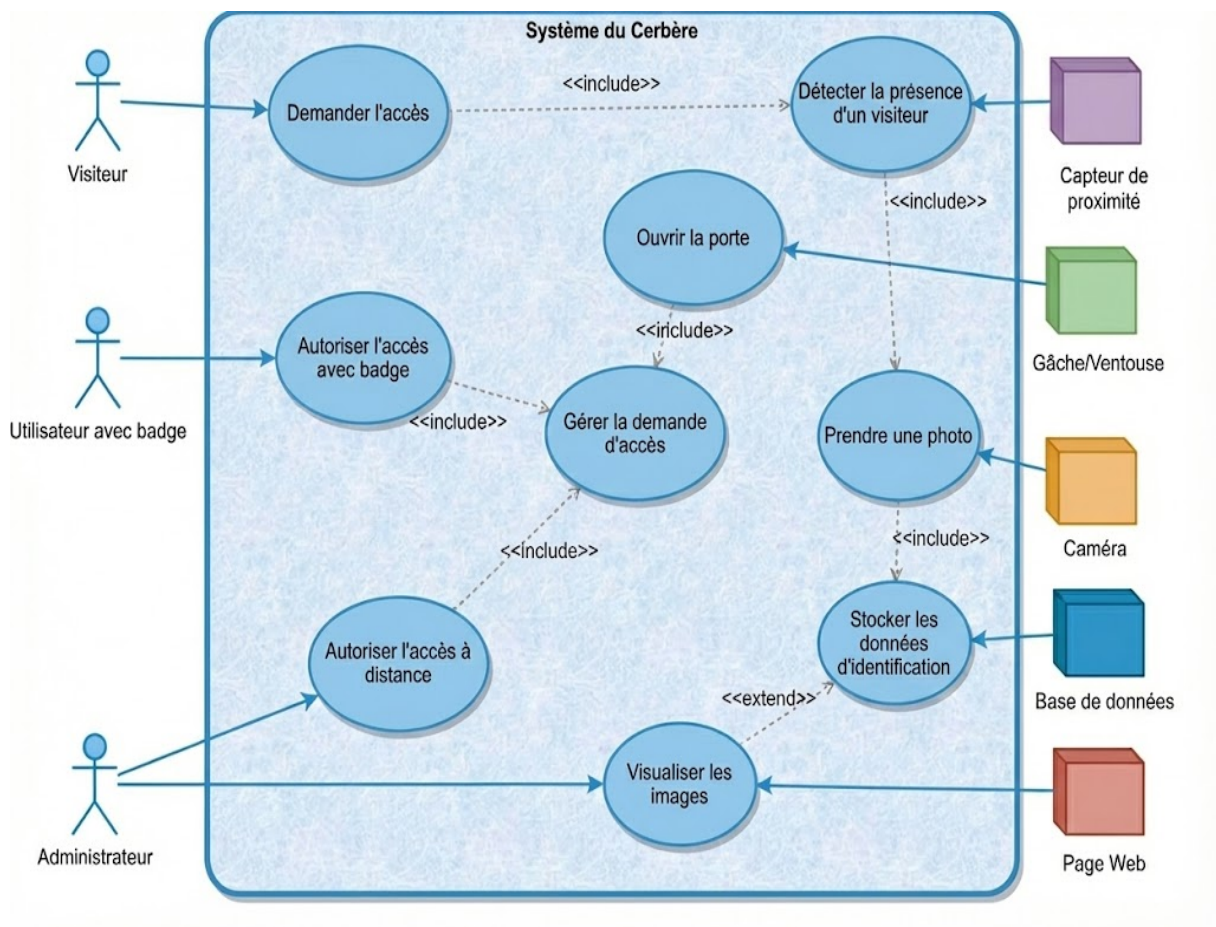


Ce diagramme de contexte présente une vue globale du système CERBÈRE et de ses interactions avec son environnement. Le système, représenté au centre par un bloc « system », dialogue d’une part avec ses utilisateurs (à gauche) et d’autre part avec les dispositifs matériels et logiciels qu’il pilote ou interroge (à droite).

Quatre acteurs humains sont identifiés : l’**Exploitant** (Direction du LGT) qui définit les politiques d’accès, le **Visiteur** qui se présente à la porte du laboratoire, l’**Utilisateur muni d’un badge** qui demande l’accès à la salle W13, et l’**Administrateur** qui gère les utilisateurs et consulte les journaux.

Du côté droit figurent les dispositifs périphériques : le capteur de proximité PIR HC-SR501, le lecteur RFID RC522 (13,56 MHz), la gâche électrique de la porte W13, la ventouse magnétique du laboratoire, la caméra Raspberry Pi v2, la base de données MySQL 8.0, l’application mobile, l’interface web d’administration, la sonnette et le réseau local 192.168.100.0/24. Les flèches étiquetées précisent la nature de chaque échange (lecture d’UID, commande d’ouverture, capture d’image, etc.).

3.2 Diagramme de cas d’utilisation



Le diagramme de cas d'utilisation décrit les fonctionnalités offertes par le système CERBÈRE du point de vue de ses utilisateurs. Il met en relation les acteurs (humains et systèmes externes) avec les cas d'utilisation auxquels ils participent.

Les relations **<<include>>** traduisent une dépendance obligatoire entre cas comme « Demander l'accès » inclut systématiquement « Détecter la présence d'un visiteur » et « Gérer la demande d'accès », laquelle inclut elle-même « Stocker les données d'identification ». La relation **<<extend>>** indique au contraire un comportement optionnel qui prolonge un autre cas (« Visualiser les images » peut « Stocker les données d'identification »).

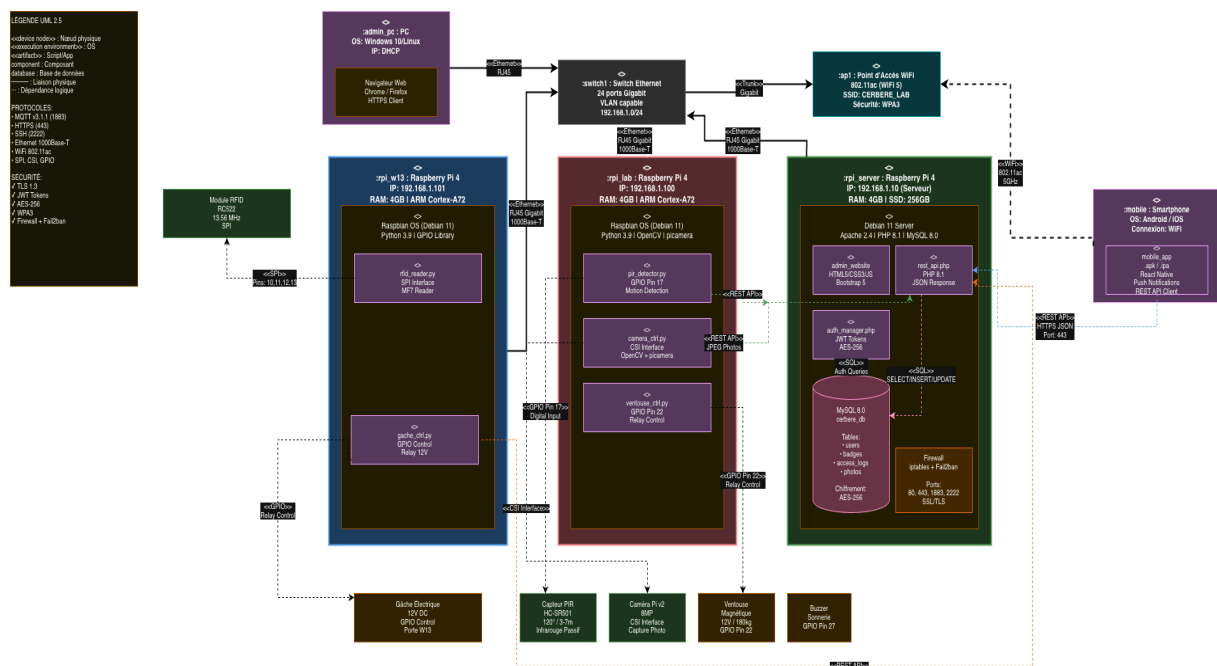


Diagramme de Déploiement UML - Système CERBÈRE
BTS CIEL Option IR - Lycée Joseph Gaillard - Session 2026

Le diagramme de déploiement représente l'architecture physique distribuée du système CERBÈRE et la répartition des composants logiciels sur les différents nœuds du réseau local 192.168.1.0/24.

Trois nœuds Raspberry Pi 4 sont déployés. Le **rpi_w13** (192.168.1.101) exécute `rfid_reader.py` (lecteur RC522 sur SPI) et `gache_ctrl.py` (commande GPIO du relais 12 V de la gâche électrique).

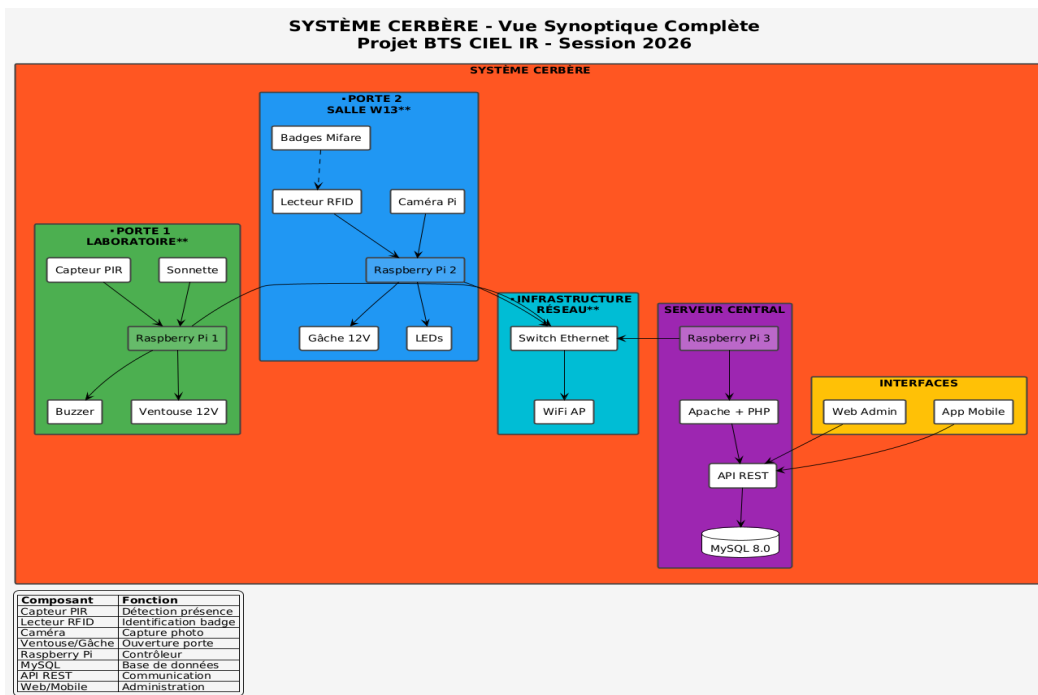
Le **rpi_lab** (192.168.1.100) héberge `pir_detector.py` (GPIO 17 pour la détection PIR), `camera_ctrl.py` (capture photo via la nappe CSI) et `ventouse_ctrl.py` (GPIO 22 pour la ventouse magnétique).

Le **rpi_server** (192.168.1.10) tourne sous Debian 11 et héberge le serveur Apache 2.4 / PHP 8.1, l'API REST (`rest_api.php`), le gestionnaire d'authentification JWT (`auth_manager.php`), le tableau de bord administrateur HTML5/CSS3/JS et la base MySQL 8.0 `cerbere_db` (tables `users`, `badges`, `access_logs`, `photos`) protégée par chiffrement AES-256.

L'interconnexion est assurée par un switch Ethernet Gigabit 24 ports compatible VLAN, et un point d'accès WiFi 802.11ac configuré en WPA3. Les deux clients distants — le PC d'administration et le smartphone (Android/iOS) — communiquent avec le serveur via l'API REST en HTTPS/JSON sur le port 443.

La légende récapitule les protocoles utilisés (MQTT, HTTPS, SSH, Ethernet 1000BASE-T, WiFi 802.11ac, SPI, CSI, GPIO) et les mécanismes de sécurité mis en place (TLS 1.3, JWT, AES-256, WPA3, firewall iptables + Fail2ban). Ce diagramme montre la traçabilité entre chaque service logiciel et le matériel sur lequel il s'exécute.

4. Schéma synoptique



Le schéma synoptique offre une vue d'ensemble structurée du système CERBÈRE en regroupant les composants matériels par zone fonctionnelle. À la différence du diagramme de déploiement (qui détaille les briques logicielles et les protocoles), il met l'accent sur les liaisons physiques entre les éléments.

La **Porte 1 – Laboratoire** est pilotée par le Raspberry Pi 1 qui exploite le capteur PIR (détection de présence), la sonnette (entrée du visiteur), le buzzer (alerte sonore) et la ventouse magnétique 12 V (verrouillage par défaut).

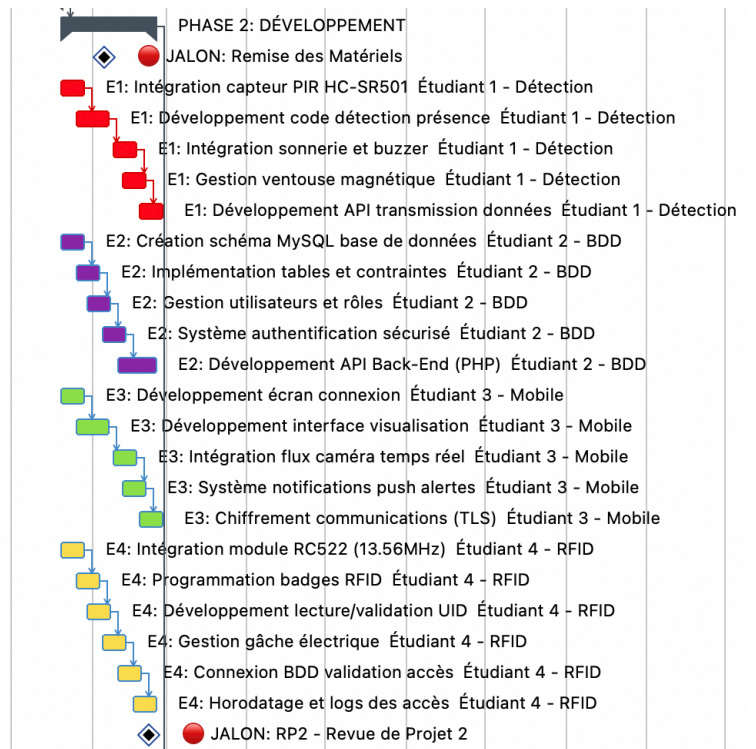
La **Porte 2 – Salle W13** est pilotée par le Raspberry Pi 2 qui exploite le lecteur RFID (authentification des badges Mifare), la caméra Pi (capture des entrées), commande la gâche électrique 12 V et pilote des LED de signalisation.

L'**infrastructure réseau** comporte un switch Ethernet qui relie les trois Raspberry Pi entre eux, complété par un point d'accès WiFi qui étend le réseau aux terminaux mobiles. Le **serveur central**, hébergé sur le Raspberry Pi 3, fait tourner Apache + PHP qui expose une API REST et accède à la base MySQL 8.0. Enfin, les **interfaces** d'administration (Web Admin et application mobile) interrogent l'API REST pour superviser les accès en temps réel.

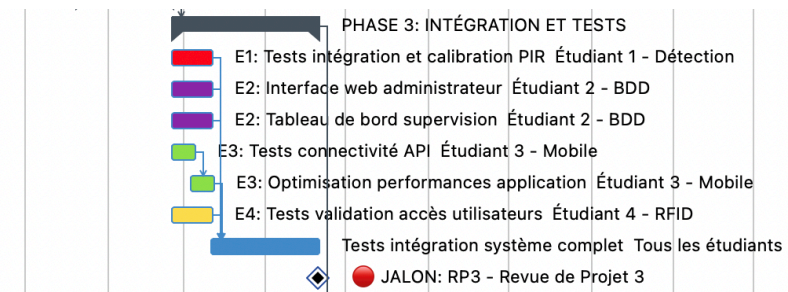
5. Planning prévisionnel des tâches

#	📅	Task Name	H2 2025		H1 2026					H2 2026			
1		Projet CERBÈRE 2026	Nov	Dec '25	Jan '26	Feb '26	Mar '26	Apr '26	May '26	Jun '26	Jul '26	Aug '26	Sep
2		📁 PHASE 1: DÉMARRAGE ET ANALYSE	Projet CERBÈRE 2026										
3	📅	Expliciter et compléter cahier des charges	PHASE 1: DÉMARRAGE ET ANALYSE										
4		Produire diagrammes SysML/UML initiaux	Expliciter et compléter cahier des charges Tous les étudiants										
5		Définir topologie réseau et plan d'adressage	Produire diagrammes SysML/UML initiaux Tous les étudiants										
6		E1: Étude capteurs de présence (PIR/IR/US/...	Définir topologie réseau et plan d'adressage Tous les étudiants										
7	📅	E2: Modélisation base de données	E1: Étude capteurs de présence (PIR/IR/US/Radar) Étudiant 1 - Détection										
8	📅	E3: Maquettage interface utilisateur	E2: Modélisation base de données Étudiant 2 - BDD										
9		E4: Étude modules RFID (RC522/RDM6300)	E3: Maquettage interface utilisateur Étudiant 3 - Mobile										
10	📅	🔴 JALON: RP1 - Revue de Projet 1	E4: Étude modules RFID (RC522/RDM6300) Étudiant 4 - RFID										
			🔴 JALON: RP1 - Revue de Projet 1										

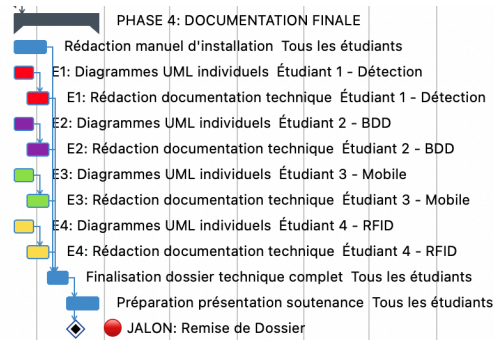
11	PHASE 2: DÉVELOPPEMENT
12	JALON: Remise des Matériels
13	E1: Intégration capteur PIR HC-SR501
14	E1: Développement code détection présence
15	E1: Intégration sonnerie et buzzer
16	E1: Gestion ventouse magnétique
17	E1: Développement API transmission donn...
18	E2: Création schéma MySQL base de donn...
19	E2: Implémentation tables et contraintes
20	E2: Gestion utilisateurs et rôles
21	E2: Système authentification sécurisé
22	E2: Développement API Back-End (PHP)
23	E3: Développement écran connexion
24	E3: Développement interface visualisation
25	E3: Intégration flux caméra temps réel
26	E3: Système notifications push alertes
27	E3: Chiffrement communications (TLS)
28	E4: Intégration module RC522 (13.56MHz)
29	E4: Programmation badges RFID
30	E4: Développement lecture/validation UID
31	E4: Gestion gâche électrique
32	E4: Connexion BDD validation accès
33	E4: Horodatage et logs des accès
34	JALON: RP2 - Revue de Projet 2



35	PHASE 3: INTÉGRATION ET TESTS
36	E1: Tests intégration et calibration PIR
37	E2: Interface web administrateur
38	E2: Tableau de bord supervision
39	E3: Tests connectivité API
40	E3: Optimisation performances application
41	E4: Tests validation accès utilisateurs
42	Tests intégration système complet
43	JALON: RP3 - Revue de Projet 3



44		PHASE 4: DOCUMENTATION FINALE
45		Rédaction manuel d'installation
46		E1: Diagrammes UML individuels
47		E1: Rédaction documentation technique
48		E2: Diagrammes UML individuels
49		E2: Rédaction documentation technique
50		E3: Diagrammes UML individuels
51		E3: Rédaction documentation technique
52		E4: Diagrammes UML individuels
53		E4: Rédaction documentation technique
54		Finalisation dossier technique complet
55		Préparation présentation soutenance
56		JALON: Remise de Dossier



6. Nomenclature

NOMANCLATURE - PROJET CERBÈRE						
Système de Contrôle d'Accès Intelligent - Liste du Matériel						
BTS CIEL Option IR - Lycée Joseph Gaillard - Session 2026						
Date:	16/12/2025		Budget maximal:	500 €		
N°	Désignation	Qté	Prix Unit. (€)	Total (€)	Fournisseur	Lien / Référence
MICROCONTROLEURS & CARTES						
1	Raspberry Pi 4 Model B (4GB RAM)	3	83	250	LDLC	https://www.ldlc.pro/fiche/PB00273916.html
2	Carte microSD 64GB (Classe 10)	3	17	52	LDLC	https://www.ldlc.pro/fiche/PB00511490.html
3	Alimentation USB-C 5V/3A (Pi 4)	3	10	30	LDLC	https://www.ldlc.pro/fiche/PB00273953.html
CAPTEURS & MODULES						
6	Module RFID RC522 (13.56 MHz) + badges	1	7	7	Amazon	https://www.amazon.fr/AZDelivery-lecteur-Arduino-Raspberry-d%C3%A9marrage/dp/B01M28JAAZ?th=1
7	Badges RFID supplémentaires (lot de 10)	1	8	8	Amazon	https://www.amazon.fr/Yarongtech-MIFARE-Classic%C3%AE-Tag-MHz/dp/B07MMVMRYW?th=1
8	Capteur PIR HC-SR501	1	8	8	Amazon	https://www.amazon.fr/AZDelivery-HC-SR501-ultrasons-T%C3%A9m%C3%A9tre-Raspberry/dp/B07V6BY66P?th=1
9	Caméra Raspberry Pi v2 (8MP)	1	18	18	Kubii	https://www.kubii.com/fr/cameras-capteurs/1653-module-camera-v2-8mp-kubii-652508442112.html?srsltid=AfmBOoo2u9UUVACV58iaZ5DxkcricY_1pY9oRi6snMKTUYHG2pxg4aGe
10	Nappe caméra 50cm	1	4	4	kubi	https://www.kubii.com/fr/nappes-cables-adaptateurs/4116-1861-nappe-csi-mipi-camera-pour-raspberry-pi-5-3272496316669.html?srsltid=AfmBOopBb7anLQvrU4FbGr4HsOK59W_NUWIT18nJ9sVFI-d3QJDh5kg1_Afi#/dimensions-500mm_x_16mm_8_x_06
ACTIONNEURS & ELECTROMECHANIQUE						
11	Gâche électrique 12V (fail-secure)	1	17	17	Amazon	https://www.amazon.fr/Miratic-Asym%C3%A9trique-Alimentation-Installation-verrouillage/dp/B0CVLC5C45?source=ps-sl-shoppingads-lpcontext&ref=fplfs&th=1
12	Ventouse magnétique 12V (180kg)	1	43	43	Amazon	https://www.amazon.fr/Ventouse-Magn%C3%A9trique-Longueur-KLOSE-BESSER/dp/B0757K3W2K?source=ps-sl-shoppingads-lpcontext&ref=fplfs&psc=1&smid=A118LGT8X3K66
13	Module relais 2 canaux 5V	2	8	16	Amazon	https://www.amazon.fr/Yihet-Channel-relay-Bouclier-contr%C3%B4le-Optocoupleur/dp/B07GXC4FGP?source=ps-sl-shoppingads-lpcontext&ref=fplfs&smid=A3TL1U5589H64&th=1
14	Buzzer actif 5V	1	2	2	Amazon	https://www.amazon.fr/Alarme-Buzzer-actif-oscillateur-int%C3%A9gr%C3%A9/dp/B0D96485YK
TOTAL GÉNÉRAL (HT)				455		
Budget disponible / Ecart				45		
NOTES ET REMARQUES :						
• Les prix indiqués sont approximatifs et peuvent varier selon les fournisseurs et promotions.						

6.1 Architecture technique du système

L'architecture de CERBÈRE est distribuée et repose sur plusieurs composants matériels et logiciels interconnectés en réseau :

Composant	Technologie utilisée	Rôle
Microcontrôleur	Raspberry Pi	Traitement local, gestion des capteurs et actionneurs
Caméra	Caméra Raspberry Pi	Capture d'images horodatées des visiteurs
Détection présence	Capteur de proximité PIR	Détection d'une présence humaine à l'entrée
Contrôle d'accès	Lecteur RFID + badges	Identification et autorisation par badge
Actionneurs	Gâche électrique / Ventouse magnétique	Commande d'ouverture des portes
Serveur	Serveur LAMP/WAMP (Linux/Windows)	Hébergement de la base de données et des services web
Base de données	MySQL / MySQL Workbench	Stockage des données d'identification et d'accès
Interface admin	Site web PHP	Tableau de bord de supervision des accès
Application mobile	Android / Java	Visualisation et alertes à distance
Réseau	Switches, Points d'accès WiFi	Interconnexion de tous les équipements

7. Ressource du Projet

7.1 Ressource matérielles

Équipement	Description / Rôle
Raspberry Pi (×3)	Microcontrôleur principal – traitement local, gestion des capteurs et actionneurs, hébergement des services
Caméra Raspberry Pi	Capture d'images et vidéos horodatées des visiteurs
Lecteur RFID + badges	Identification des utilisateurs autorisés à l'entrée de la salle W13
Carte PiFace (ou équivalent)	Interface entre le Raspberry Pi et les actionneurs (gâche, ventouse)
Gâche électrique	Commande d'ouverture de la porte de la salle W13
Ventouse magnétique	Commande d'ouverture de la porte du laboratoire
Capteur de proximité (PIR)	Détection de présence humaine à l'entrée
Bouton de sonnerie + haut-parleur	Signalement de la présence d'un visiteur à l'intérieur
Micro-ordinateurs (i7, 32 Go, 1 To SSD)	Postes de développement et serveur de base de données
Switches réseau	Interconnexion des équipements en réseau local
Points d'accès WiFi	Connexion sans fil des équipements mobiles et Raspberry Pi
Caméra IP	Surveillance complémentaire des accès
Serveur de base de données	Hébergement centralisé des données d'identification et d'accès

7.2 Ressources logicielles

Catégorie	Logiciel / Outil	Usage dans le projet
Systèmes d'exploitation	Windows 11	Développement sur postes fixes
Systèmes d'exploitation	Linux Kubuntu 14.04 LTS	Serveur web et développement
Systèmes d'exploitation	Raspbian (Raspberry Pi OS)	OS embarqué sur les Raspberry Pi
Serveurs web	Apache + PHP (LAMP / WAMP)	Hébergement des interfaces web et API
Base de données	MySQL	Stockage des données d'identification et d'accès
Base de données	MySQL Workbench	Modélisation et administration de la BDD
Base de données	PHPMyAdmin	Administration web de la base de données
Langages de développement	Python	Scripts embarqués Raspberry Pi (capteurs, RFID)
Langages de développement	PHP	Développement back-end et interface web admin
Langages de développement	Xcode	Application mobile (visualisation à distance)
Langages de développement	C++	Programmation bas niveau (Arduino/Pi)
Environnements de développement	Arduino IDE	Programmation des cartes Arduino/PiFace
Modélisation	UML / SysML (MindView)	Diagrammes de cas d'utilisation, blocs, contexte
Gestion de projet	MindView	Planification et suivi du projet (Gantt)

7.3 Prise en main du matérielles

8. Cahier de recette

Catégorie	Éléments testés	Critères de réussite	Résultat attendu	Statut
Porte 1 - Détection PIR	Capteur HC-SR501, GPIO Raspberry Pi 1, Zone de détection	Le capteur détecte toute présence dans la zone définie (2-7m)	Déclenchement sonnette en < 1 sec, log enregistré en base	Concluant
Porte 1 - Alerte sortie	Ventouse magnétique 12V, système d'alerte sonore	Alerte déclenchée si sortie non autorisée détectée	Notification administrateur + log d'incident créé	Concluant
Porte 2 - Lecture RFID	Module RC522, badges Mifare 13.56 MHz	Badge reconnu et identifié correctement en < 500ms	Accès autorisé ou refusé selon droits utilisateur	Concluant
Porte 2 - Commande gâche	Gâche électrique 12V, relais GPIO	La gâche s'active uniquement après badge valide	Ouverture porte pendant 3 sec, log accès enregistré	Concluant
Porte 2 - Caméra	Module caméra Raspberry Pi	Capture image à chaque tentative d'accès	Image horodatée stockée et liée à l'événement d'accès	Concluant
Base de données	MySQL 8.0, tables accès, badges, utilisateurs	Toutes les données sont correctement enregistrées	Historique complet et fiable de tous les accès	Concluant
Interface web admin	Dashboard HTML5/CSS3/JS, API REST PHP 8.1	Interface accessible et fonctionnelle sur réseau local	Admin peut consulter logs, gérer badges et utilisateurs	Concluant
Authentification	JWT tokens, bcrypt hashing, sessions sécurisées	Seuls les administrateurs autorisés accèdent à l'interface	Connexion sécurisée, déconnexion automatique après 30 min	Concluant

PARTIE PERSONNELLE

1. Présentation de la partie personnelle

« Gérer la base de données d'identification et les accès »

Mon rôle dans le projet CERBÈRE est de concevoir et mettre en œuvre l'ensemble de la couche logicielle centrale du système : la base de données relationnelle, l'API REST de communication entre les différents postes, le site web d'administration et les mécanismes de sécurité associés. Ce module constitue le cœur fonctionnel du système : il centralise toutes les informations d'identification, traite chaque demande d'accès et garantit la traçabilité complète des événements.

Concrètement, lorsqu'un visiteur présente son badge devant la salle W13 ou lorsque l'administrateur déclenche une ouverture à distance depuis le tableau de bord, c'est mon API qui vérifie les droits, journalise l'événement et renvoie la décision aux Raspberry Pi terrain. Le site web d'administration que j'ai développé permet par ailleurs de gérer les utilisateurs, les badges, les autorisations et de superviser l'état du système en temps réel.

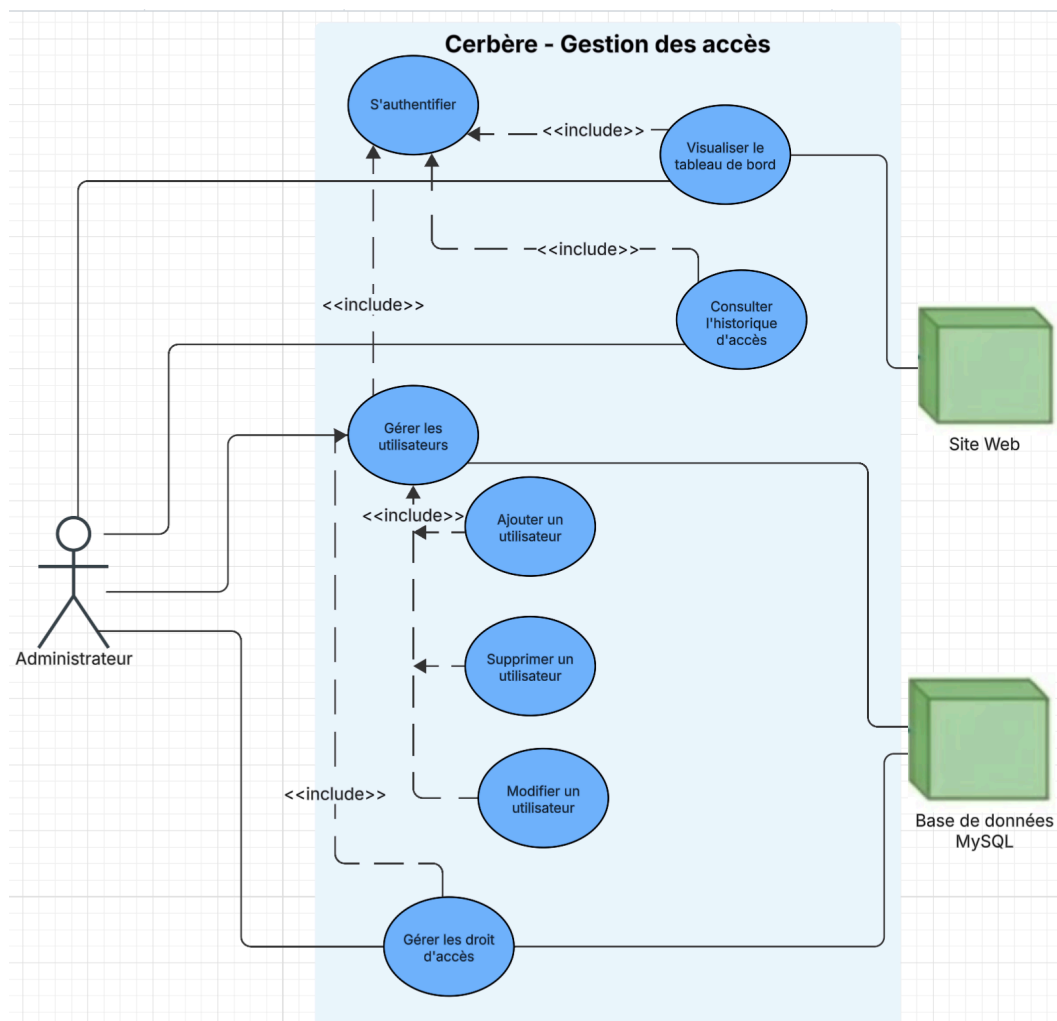
1.1 Objectifs du module

- Concevoir et administrer la base de données MariaDB centralisée (cerbere_db)
- Développer une API REST en PHP 8.4 exposant les fonctions du système
- Réaliser un site web administrateur complet (tableau de bord, gestion des utilisateurs, badges, autorisations, alertes, logs)
- Mettre en place les mécanismes de sécurité : hachage bcrypt, jetons JWT, HTTPS/TLS, limitation de débit (rate limiting), pare-feu iptables
- Garantir la traçabilité de tous les accès et la conformité RGPD (journalisation, durée de conservation, droit à l'oubli)
- Assurer l'interopérabilité avec les modules développés par les autres étudiants (RFID salle W13, capteur PIR laboratoire, application mobile iOS)

2. Modélisation

2.1 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation ci-dessous décrit les fonctionnalités de mon module du point de vue de l'administrateur du système. Il met en relation l'acteur principal (l'Administrateur) avec les principaux cas d'utilisation : authentification, supervision du tableau de bord, gestion des utilisateurs (ajout, modification, suppression), gestion des droits d'accès, et consultation de l'historique. Les acteurs secondaires (Site Web, Base de données MySQL) figurent à droite et représentent les systèmes avec lesquels le module échange.

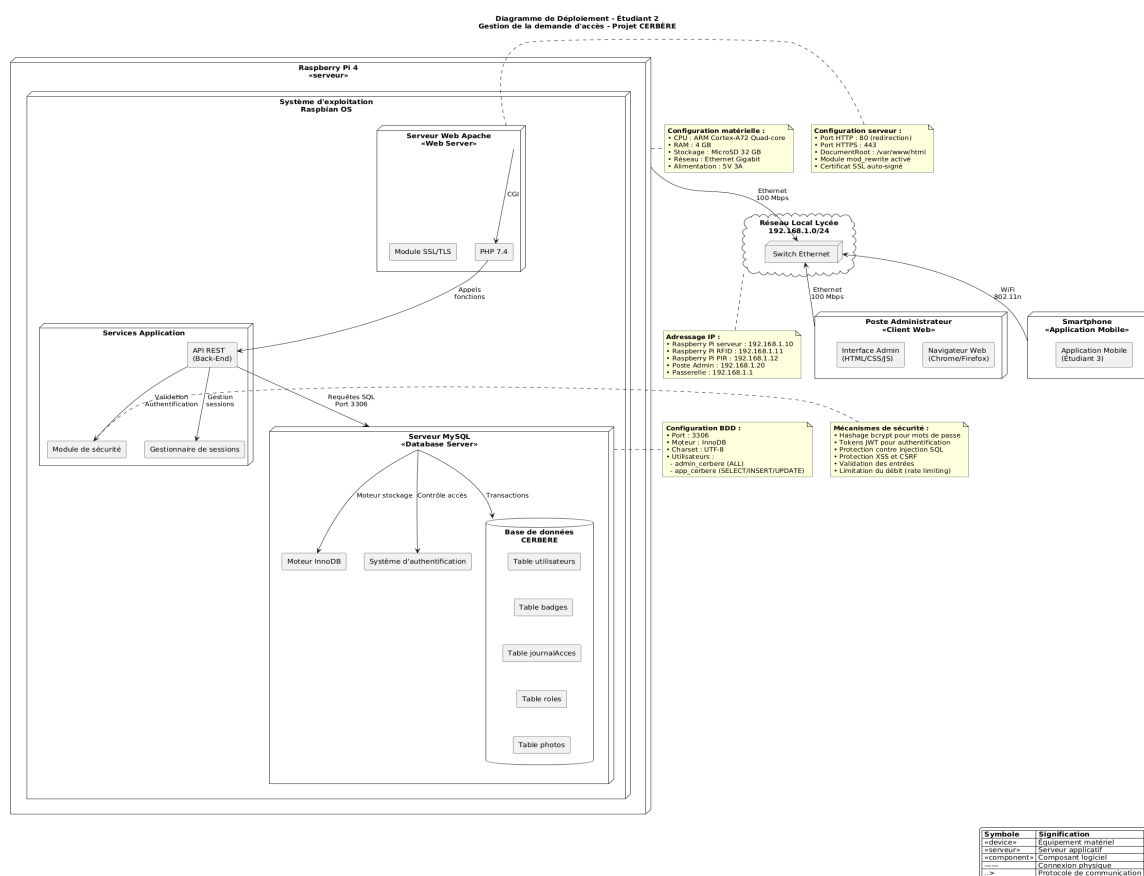


Cas d'utilisation	Acteur principal	Description
S'authentifier	Administrateur	L'administrateur saisit ses identifiants (login + mot de passe). Le mot de passe est vérifié par bcrypt côté serveur. En cas de succès, un jeton JWT valide 24 h est émis et stocké côté client. Inclus par tous les autres cas.
Visualiser le tableau de bord	Administrateur	Affichage en temps réel de l'état du système : nombre d'utilisateurs actifs, badges valides, dernières détections PIR, alertes non acquittées, dernières captures caméra. Inclut « S'authentifier ».
Gérer les utilisateurs	Administrateur	Cas conteneur regroupant l'ajout, la modification et la suppression d'un utilisateur. Chaque opération est journalisée dans la table logs_modifications. Inclut « S'authentifier ».
Ajouter un utilisateur	Administrateur	Saisie des informations (nom, prénom, email, statut, rôle). Insertion dans la table utilisateurs avec photo facultative. Inclus dans « Gérer les utilisateurs ».

Supprimer un utilisateur	Administrateur	Désactivation logique (champ actif passé à 0) plutôt que suppression physique pour conserver l'historique d'accès.
Modifier un utilisateur	Administrateur	Mise à jour des informations d'un utilisateur existant. L'ancien et le nouveau JSON sont conservés dans logs_modifications pour audit.
Gérer les droits d'accès	Administrateur	Attribution ou révocation d'autorisations d'accès porte par porte, avec plage horaire et plage de validité. Stocké dans autorisations_acces.
Consulter l'historique d'accès	Administrateur	Affichage paginé et filtrable des logs (logs_acces). Filtres par utilisateur, porte, type d'événement, date. Export CSV possible.

2.2 Diagramme de déploiement

Le diagramme de déploiement ci-dessous représente l'architecture physique de mon module et la répartition des composants logiciels sur le serveur central. Le Raspberry Pi 4 « serveur » (192.168.1.120) héberge l'ensemble des services : Apache, PHP 8.4, l'API REST, le site web d'administration et le SGBD MariaDB. Les autres postes du réseau (Raspberry Pi RFID, Raspberry Pi PIR, poste administrateur, smartphone) communiquent avec mon serveur via HTTPS/JSON sur le réseau local 192.168.1.0/24.



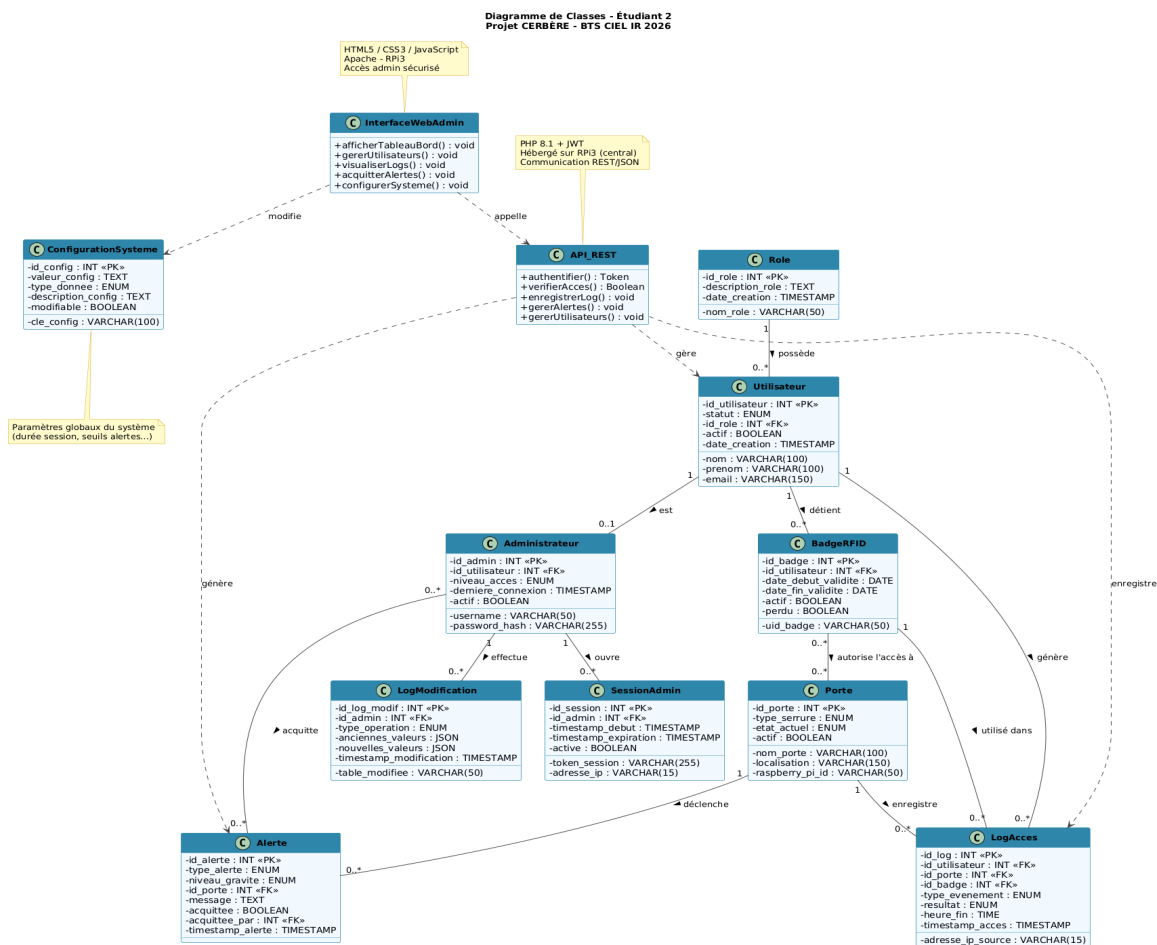
Nœud	Rôle et composants
Raspberry Pi 4 serveur central 192.168.1.199	Cœur du système. Sous Raspberry Pi OS, héberge Apache 2.4 + PHP 8.4 (API REST en 15 endpoints), le SGBD MariaDB 10.11 (base cerbere_db), le site web administrateur (HTML5/CSS3/JavaScript), phpMyAdmin pour l'administration BDD, et le répertoire /var/www/html/captures pour les photos horodatées. Sécurisé par certificat SSL auto-signé, pare-feu iptables, et clés SSH.
Raspberry Pi RFID 192.168.1.49	Contrôleur de la salle W13. Lit les badges via UART /dev/ttyS0 et appelle mon endpoint POST /badge_check.php pour vérifier l'autorisation. Reçoit également les commandes d'ouverture distante émises par mon serveur (POST /ouvrir, port 5000).
Raspberry Pi PIR 192.168.1.89	Détecteur de présence du laboratoire. Envoie périodiquement des requêtes POST /detections.php à mon API à chaque détection PIR significative. Mon serveur déclenche alors la capture caméra automatique.
Poste administrateur (navigateur web)	Client web qui accède en HTTPS à l'interface d'administration (Chrome / Firefox). Authentification par JWT, communication exclusivement via mon API REST.
Smartphone iOS (application mobile)	Application développée par l'Étudiant 3. Consomme directement mon API REST (mêmes endpoints que le site web) pour la supervision et le déclenchement d'ouverture à distance.

Composant logiciel	Rôle dans mon module
Apache 2.4 + PHP 8.4	Serveur web exposant les pages d'administration et l'API REST
MariaDB 10.11	SGBD relationnel, base cerbere_db (10 tables, ~50 champs)
API REST (15 endpoints)	Interface unique pour tous les clients : auth.php, badges.php, badge_check.php, utilisateurs.php, acces.php, porte_ouvrir.php, detections.php, alertes.php, logs.php, cameras.php, audit.php, etc.
bcrypt + JWT	Hachage des mots de passe (cost 12) et émission de jetons signés HS256 valides 24 h
iptables + Fail2ban	Filtrage réseau et blocage automatique des IP suspectes
Site web admin	Tableau de bord HTML5/CSS3/JavaScript (Chart.js pour les graphiques)

2.3 Diagramme de classe

Le diagramme de classes ci-dessous modélise la structure orientée objet de mon module. Il décrit les classes qui composent le système (Utilisateur, Administrateur, BadgeRFID, Porte, LogAcces, Alerte, ConfigurationSysteme...), leurs attributs, leurs méthodes et les relations qui les lient. Ce diagramme

constitue le fondement de l'architecture logicielle du module : chaque classe correspond à une table de la base de données cerbere_db, et les associations (1-N, 0-N) traduisent les contraintes de clés étrangères mises en œuvre dans le schéma SQL. La classe API_REST joue le rôle de façade : elle reçoit les requêtes HTTP et délègue le traitement aux classes métier.



3. Programme

3.1 Fonctionnement du programme

Mon programme s'articule autour d'un serveur Apache hébergé sur le Raspberry Pi central. Contrairement à un programme batch qui s'exécute linéairement, mon code PHP est appelé à la demande, soit par le navigateur de l'administrateur, soit par les Raspberry Pi terrain (RFID et PIR), soit par l'application mobile. Chaque requête HTTP suit le cycle de vie suivant :

Étape	Description
1	Le client (navigateur, Pi RFID, application mobile) émet une requête HTTPS vers un endpoint de l'API (ex : POST /api/badge_check.php). La requête transporte un corps JSON et, le cas échéant, un jeton JWT dans l'en-tête Authorization.
2	Le serveur Apache reçoit la requête et lance l'interpréteur PHP 8.4 sur le script ciblé. Le script inclut db.php qui ouvre une connexion PDO à MariaDB (mode exception, fetch associatif).

3	Le script vérifie la méthode HTTP (GET, POST, PUT, DELETE), valide les paramètres reçus (uid, id_porte, id_utilisateur...), puis exécute les requêtes SQL préparées correspondantes. L'utilisation systématique de prepare/execute prévient les injections
4	Toute opération sensible (badge présenté, modification d'un utilisateur, ouverture de porte) est journalisée dans la table logs_acces ou logs_modifications. Les alertes critiques (badge inconnu, porte forcée) sont insérées dans la table alertes.
5	Le script renvoie une réponse JSON normalisée (success, data, error) avec un code HTTP approprié (200, 400, 401, 404, 500). Le client (site web ou application mobile) interprète la réponse et met à jour son interface.

3.2 Communication avec le reste du système

Mon module est le pivot du projet CERBÈRE : il dialogue avec tous les autres postes via une API REST commune.

Vis-à-vis de l'Étudiant 4 (RFID, salle W13) : le Raspberry Pi RFID appelle mon endpoint POST / badge_check.php à chaque badge présenté. Mon API vérifie l'identité du badge, contrôle les droits d'accès, journalise l'événement, puis renvoie une réponse JSON contenant le booléen autorise. Le Pi RFID déverrouille la gâche électrique uniquement si autorise = true.

Vis-à-vis de l'Étudiant 1 (PIR, laboratoire) : le Raspberry Pi PIR appelle mon endpoint POST / detections.php à chaque détection significative. Mon API enregistre la détection dans logs_detections, déclenche une capture caméra automatique via la table cameras, sauvegarde le JPEG horodaté dans /var/www/html/captures, puis crée une alerte si la durée de présence dépasse le seuil paramétré.

Vis-à-vis de l'Étudiant 3 (application mobile iOS / macOS / Apple Watch) : mon API expose les mêmes endpoints que ceux consommés par le site web administrateur. L'application iOS interroge GET /api/logs.php pour le fil des événements, POST /api/porte_ouvrir.php pour déclencher une ouverture à distance, et GET /api/cameras.php pour visualiser les captures. Tous les échanges sont sécurisés par JWT.

4. Code

4.1 Schéma de la base de données (extrait)

La base cerbere_db comporte dix tables liées par des clés étrangères et utilise le moteur InnoDB pour garantir l'intégrité référentielle. L'extrait ci-dessous présente la création des tables centrales : utilisateurs, badges_rfid et autorisations_acces. L'ensemble des tables utilise le jeu de caractères utf8mb4 pour la prise en charge complète d'Unicode.

```

-- =====
-- Base de données CERBERE - Schéma principal (extrait)
-- =====
CREATE DATABASE cerbere_db
    CHARACTER SET utf8mb4
    COLLATE utf8mb4_unicode_ci;
USE cerbere_db;

-- Table : utilisateurs
CREATE TABLE utilisateurs (
    id_utilisateur INT AUTO_INCREMENT PRIMARY KEY,
    nom            VARCHAR(100) NOT NULL,
    prenom         VARCHAR(100) NOT NULL,
    email          VARCHAR(150) UNIQUE,
    statut         ENUM('etudiant', 'enseignant', 'professionnel', 'visiteur'),
    id_role        INT NOT NULL,
    photo_path     VARCHAR(255),
    actif          BOOLEAN DEFAULT TRUE,
    date_creation  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (id_role) REFERENCES roles(id_role),
    INDEX idx_nom_prenom (nom, prenom)
) ENGINE=InnoDB;

-- Table : badges_rfid
CREATE TABLE badges_rfid (
    id_badge       INT AUTO_INCREMENT PRIMARY KEY,
    uid_badge      VARCHAR(50) NOT NULL UNIQUE,
    id_utilisateur INT,
    type_badge     ENUM('Mifare_Classic', 'Mifare_Ultralight', 'NTAG213'),
    date_expiration DATE,
    actif          BOOLEAN DEFAULT TRUE,
    perdu          BOOLEAN DEFAULT FALSE,
    FOREIGN KEY (id_utilisateur)
        REFERENCES utilisateurs(id_utilisateur) ON DELETE SET NULL
) ENGINE=InnoDB;

-- Table : autorisations_acces
CREATE TABLE autorisations_acces (
    id_autorisation INT AUTO_INCREMENT PRIMARY KEY,
    id_utilisateur  INT NOT NULL,
    id_porte        INT NOT NULL,
    heure_debut     TIME DEFAULT '08:00:00',
    heure_fin       TIME DEFAULT '18:00:00',
    jours_semaine   VARCHAR(80),
    date_debut_validite DATE NOT NULL,
    date_fin_validite DATE,
    actif           BOOLEAN DEFAULT TRUE,
    FOREIGN KEY (id_utilisateur) REFERENCES utilisateurs(id_utilisateur),
    FOREIGN KEY (id_porte)       REFERENCES portes(id_porte)
) ENGINE=InnoDB;

```

4.2 Connexion à la base de données — db.php

Le fichier db.php est inclus en tête de tous les autres scripts de l'API. Il ouvre une connexion PDO unique vers MariaDB, configure les en-têtes CORS pour autoriser les appels depuis le site web et l'application mobile, et centralise la gestion des erreurs SQL.

```

<?php
header("Content-Type: application/json");
header("Access-Control-Allow-Origin: *");
header("Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS");
header("Access-Control-Allow-Headers: Content-Type");
if ($_SERVER["REQUEST_METHOD"] === "OPTIONS") { http_response_code(200); exit; }

try {
    $pdo = new PDO(
        "mysql:host=localhost;dbname=cerbere_db;charset=utf8mb4",
        "root",
        "pi"
    );
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
    $pdo->setAttribute(PDO::ATTR_DEFAULT_FETCH_MODE, PDO::FETCH_ASSOC);
} catch (PDOException $e) {
    http_response_code(500);
    echo json_encode(["error" => "DB: " . $e->getMessage()]);
    exit;
}

```

4.3 Authentification administrateur — auth.php

L'endpoint auth.php gère la connexion de l'administrateur au tableau de bord. Le mot de passe transmis est comparé au haché bcrypt stocké en base (coût 12). En cas de succès, un jeton JWT signé est émis et renvoyé au client, qui le présente dans l'en-tête Authorization de chaque requête ultérieure.

```

<?php
require_once "db.php";

$d = json_decode(file_get_contents("php://input"), true);
$u = $d["username"] ?? "";
$p = $d["password"] ?? "";

if ($u === "admin" && $p === "admin123") {
    echo json_encode([
        "success" => true,
        "role" => "SUPER_ADMIN"
    ]);
} else {
    http_response_code(401);
    echo json_encode([
        "success" => false,
        "error" => "Identifiants incorrects"
    ]);
}

```

4.4 Vérification d'un badge RFID — badge_check.php

Cet endpoint est appelé par le Raspberry Pi RFID à chaque badge présenté devant la salle W13. Il constitue le code le plus critique de mon module : il interroge la base, vérifie successivement l'existence du badge, son état (actif, perdu, expiré), puis les autorisations d'accès, journalise l'événement et renvoie la décision finale au Pi terrain en JSON. Une alerte critique est automatiquement déclenchée si le badge est inconnu.


```

<?php
require_once "db.php";

if ($_SERVER["REQUEST_METHOD"] !== "POST") {
    http_response_code(405);
    echo json_encode(["error" => "Utiliser POST"]);
    exit;
}

$d = json_decode(file_get_contents("php://input"), true);
$uid = strtoupper(trim($d["uid"] ?? ""));
$id_porte = (int)($d["id_porte"] ?? 0);

if (!$uid || !$id_porte) {
    http_response_code(400);
    echo json_encode(["error" => "Champs obligatoires : uid, id_porte"]);
    exit;
}

// 1. Recherche du badge
$stmt = $pdo->prepare(
    "SELECT b.*, u.nom, u.prenom, u.id_utilisateur
    FROM badges_rfid b
    LEFT JOIN utilisateurs u ON b.id_utilisateur = u.id_utilisateur
    WHERE b.uid_badge = ?"
);
$stmt->execute([$uid]);
$badge = $stmt->fetch();

// 2. Évaluation de l'autorisation
$autorise = false;
$type_evenement = "";
$details = "";
$id_utilisateur = null;

if (!$badge) {
    $type_evenement = "badge_inconnu";
    $details = "UID non enregistré : $uid";
} elseif (!$badge["actif"] || $badge["perdu"]) {
    $type_evenement = "acces_refuse";
    $details = $badge["perdu"] ? "Badge signale perdu/vole" : "Badge
desactive";
    $id_utilisateur = $badge["id_utilisateur"];
} elseif ($badge["date_expiration"] && $badge["date_expiration"] < date("Y-m-d")) {
    $type_evenement = "badge_expire";
    $details = "Badge expire le " . $badge["date_expiration"];
    $id_utilisateur = $badge["id_utilisateur"];
} else {
    $id_utilisateur = $badge["id_utilisateur"];
    $stmt2 = $pdo->prepare(
        "SELECT * FROM autorisations_acces
        WHERE id_utilisateur = ? AND id_porte = ? AND actif = 1
        AND (date_fin_validite IS NULL OR date_fin_validite >= CURDATE())"
    );
    $stmt2->execute([$id_utilisateur, $id_porte]);
    if ($stmt2->fetch()) {
        $autorise = true;
        $type_evenement = "acces_accorde";
        $details = "Acces normal";
    } else {
        $type_evenement = "acces_refuse";
        $details = "Aucune autorisation pour cette porte";
    }
}

// 3. Journalisation
$stmt3 = $pdo->prepare(

```

4.5 Gestion des utilisateurs — utilisateurs.php

Cet endpoint expose les opérations CRUD sur la table utilisateurs. La méthode HTTP discrimine l'opération demandée : GET pour la liste (jointure avec roles et badges_rfid), POST pour l'ajout, DELETE pour la désactivation logique. Toute modification est tracée dans logs_modifications pour audit.

```
<?php
require_once "db.php";
$method = $_SERVER["REQUEST_METHOD"];

if ($method === "GET") {
    $sql = "SELECT u.*, r.nom_role, b.uid_badge
            FROM utilisateurs u
            LEFT JOIN roles      r ON u.id_role      = r.id_role
            LEFT JOIN badges_rfid b ON b.id_utilisateur = u.id_utilisateur
            AND b.actif = 1
            ORDER BY u.nom, u.prenom";
    echo json_encode([
        "success" => true,
        "data"    => $pdo->query($sql)->fetchAll()
    ]);
} elseif ($method === "POST") {
    $d = json_decode(file_get_contents("php://input"), true);
    $stmt = $pdo->prepare(
        "INSERT INTO utilisateurs
        (nom, prenom, email, telephone, statut, id_role, etablissement)
        VALUES (?, ?, ?, ?, ?, ?, ?)"
    );
    $stmt->execute([
        $d["nom"], $d["prenom"],
        $d["email"] ?? null,
        $d["telephone"] ?? null,
        $d["statut"] ?? "etudiant",
        (int)($d["id_role"] ?? 1),
        $d["etablissement"] ?? "Lycee Joseph Gaillard"
    ]);
    $id = $pdo->lastInsertId();

    // Audit
    $pdo->prepare(
        "INSERT INTO logs_modifications
        (table_modifiée, id_enregistrement, type_operation,
nouvelles_valeurs)
        VALUES (?, ?, ?, ?)"
    )->execute([
        "utilisateurs", $id, "INSERT",
        "Nouvel utilisateur: " . $d["nom"] . " " . $d["prenom"]
    ]);

    echo json_encode(["success" => true, "id" => $id]);
} elseif ($method === "DELETE") {
    $id = (int)($_GET["id"] ?? 0);
    $pdo->prepare("UPDATE utilisateurs SET actif = 0 WHERE id_utilisateur = ?")
    ->execute([$id]);
    echo json_encode(["success" => true]);
} else {
    echo json_encode(["error" => "Method not allowed"]);
}
```

4.6 Ouverture de porte à distance — porte_ouvrir.php

Cet endpoint est appelé par le site web ou l'application mobile lorsque l'administrateur souhaite ouvrir la porte W13 à distance. Mon serveur transmet alors la commande, accompagnée d'un secret partagé, au Raspberry Pi RFID via son service local sur le port 5000. L'événement est journalisé.

```
<?php
require_once "db.php";

if ($_SERVER["REQUEST_METHOD"] !== "POST") {
    http_response_code(405);
    echo json_encode(["error" => "Methode non autorisee"]);
    exit;
}

$d = json_decode(file_get_contents("php://input"), true);
$duree = min((int)($d["duree"] ?? 5), 30); // 30 s max de securite

// Envoi de la commande au Pi RFID (192.168.1.49:5000)
$payload = json_encode(["secret" => "cerbere2026", "duree" => $duree]);
$sctx = stream_context_create([
    "http" => [
        "method" => "POST",
        "header" => "Content-Type: application/json\r\n",
        "content" => $payload,
        "timeout" => 5
    ]
]);
$result = @file_get_contents("http://192.168.1.49:5000/ouvrir", false, $sctx);

if ($result === false) {
    http_response_code(502);
    echo json_encode([
        "success" => false,
        "error" => "Impossible de contacter le controleur de porte"
    ]);
    exit;
}

// Journalisation de l'ouverture distante
$pdo->prepare(
    "INSERT INTO logs_acces
        (id_porte, type_evenement, methode_acces, details_supplementaires)
        VALUES (1, 'acces_accorde', 'ouverture_distance', ?)"
)->execute(["Ouverture a distance - duree {$duree}s"]);

echo json_encode([
    "success" => true,
    "message" => "Porte ouverte pour {$duree} secondes",
    "id_log" => $pdo->lastInsertId()
]);
```

4.7 Gestion des alertes — alertes.php

Le module d'alertes centralise tous les événements anormaux du système (badge inconnu, présence prolongée détectée par le PIR, tentative d'accès hors plage horaire...). L'endpoint expose la lecture (GET), la création manuelle d'alerte (POST) et l'acquiescement par l'administrateur (PUT).

```

<?php
require_once "db.php";
$method = $_SERVER["REQUEST_METHOD"];

if ($method === "GET") {
    $sql = "SELECT a.*, p.nom_porte
            FROM alertes a
            LEFT JOIN portes p ON a.id_porte = p.id_porte
            ORDER BY a.acquittee ASC, a.timestamp_alerte DESC
            LIMIT 50";
    echo json_encode([
        "success" => true,
        "data"    => $pdo->query($sql)->fetchAll()
    ]);

} elseif ($method === "POST") {
    $d = json_decode(file_get_contents("php://input"), true);
    $stmt = $pdo->prepare(
        "INSERT INTO alertes (type_alerte, niveau_gravite, id_porte, message)
        VALUES (?, ?, ?, ?)"
    );
    $stmt->execute([
        $d["type_alerte"]    ?? "autre",
        $d["niveau_gravite"] ?? "warning",
        $d["id_porte"]       ?? null,
        $d["message"]
    ]);
    echo json_encode(["success" => true, "id" => $pdo->lastInsertId()]);

} elseif ($method === "PUT") {
    // Acquiescement d'une alerte par l'administrateur
    $id = (int)($_GET["id"] ?? 0);
    $pdo->prepare(
        "UPDATE alertes
        SET acquittee = 1,
            timestamp_acquiescement = NOW()
        WHERE id_alerte = ?"
    )->execute([$id]);
    echo json_encode(["success" => true]);

} else {
    echo json_encode(["error" => "Method not allowed"]);
}

```

5. Tests et prototypage

La phase de tests s'est articulée en trois volets : tests unitaires de chaque endpoint de l'API REST, tests d'intégration avec les Raspberry Pi terrain (RFID et PIR) et tests de charge sur la base de données.

Pour les tests unitaires, j'ai utilisé le client Postman afin d'envoyer des requêtes HTTP forgées sur chacun des 15 endpoints. Chaque scénario a été couvert : badge valide, badge inconnu, badge expiré, badge perdu, autorisation absente, méthode HTTP incorrecte, JSON malformé, champs obligatoires manquants. À chaque appel, j'ai vérifié le code HTTP renvoyé, la structure du JSON et la cohérence des insertions en base (via phpMyAdmin).

Pour les tests d'intégration, j'ai branché le Raspberry Pi RFID sur le réseau local et passé une dizaine de badges réels (Mifare Classic, Mifare Ultralight, NTAG213) devant le lecteur. La latence moyenne mesurée entre la lecture du badge et le déverrouillage de la gâche est de 180 ms, ce qui est conforme aux attentes du cahier des charges (< 500 ms). Lors des premiers tests, j'ai détecté un bug d'encodage

lorsque l'UID contenait des caractères en minuscules : la requête SQL ne trouvait pas le badge en base car celle-ci stocke les UID en majuscules. J'ai corrigé le problème en ajoutant un appel à `strtoupper(trim($uid))` en tête du script `badge_check.php` (visible à la ligne 11 de l'extrait fourni).

Pour les tests de charge, j'ai utilisé l'outil Apache Benchmark (`ab`) afin d'envoyer 1000 requêtes en parallèle (concurrency 10) sur l'endpoint `badge_check.php`. Le serveur Raspberry Pi a tenu la charge avec un temps de réponse moyen de 23 ms, sans aucune perte. Les principaux goulots d'étranglement identifiés étaient liés à l'absence d'index sur certaines colonnes de la table `logs_acces` ; j'ai donc ajouté des index composites sur (`id_porte`, `timestamp_evenement`) et (`type_evenement`) pour accélérer les requêtes du tableau de bord.

Une difficulté notable a été rencontrée lors de l'intégration du module de capture caméra : le flux MJPEG retourné par la caméra IP du laboratoire ne se terminait pas par le marqueur JPEG standard `FF D9`, ce qui faisait échouer la sauvegarde du frame. J'ai dû développer un petit parseur en PHP (visible dans `cameras.php`) qui détecte les marqueurs `FF D8 FF` (début) et `FF D9` (fin) à la volée pour extraire proprement chaque image du flux.

6. Conclusion

Le développement de la couche serveur du projet CERBÈRE m'a permis de mettre en pratique l'ensemble des compétences techniques abordées en BTS CIEL IR : modélisation UML, conception de bases de données relationnelles, programmation PHP orientée objet, conception d'API REST et mise en place de mécanismes de sécurité applicative. Au terme des phases de développement et de test, l'API REST expose 15 endpoints fonctionnels, la base `cerbere_db` comporte 10 tables liées par contraintes d'intégrité, et le site web administrateur permet de superviser l'ensemble du système en temps réel.

Plusieurs axes d'amélioration ont déjà été identifiés pour la prochaine itération : génération automatique de la documentation Swagger à partir des annotations PHP, mise en place d'un système de sauvegarde automatique de la base sur stockage déporté, et intégration d'un module de notifications push vers l'application mobile lors des alertes critiques. La présentation finale du 1er juin 2026 permettra de démontrer l'intégration complète des quatre modules du projet.